

Bus Cycle

A **bus cycle** is the fundamental unit of data transfer between the central processing unit (CPU), memory, and peripheral devices over a system bus. It represents the sequence of operations required to complete a single transaction on the bus, such as reading from or writing to memory or I/O devices.

Key Components of a Bus Cycle:

1. **Address Phase:**
 - The CPU or bus master places the memory or I/O address on the address bus.
 - This phase specifies the location for the data transfer.
2. **Control Phase:**
 - The control signals (e.g., read/write, memory or I/O access, byte/word size) are activated to indicate the type of transaction.
3. **Data Phase:**
 - Data is transferred between the source and destination using the data bus.
 - It can be a read operation (memory to CPU) or a write operation (CPU to memory).
4. **Acknowledgment:**
 - The receiving device sends a signal (such as an ACK) to confirm the successful completion of the transfer.

Types of Bus Cycles

1. **Memory Read Cycle:**
 - The CPU reads data from a specified memory location.
2. **Memory Write Cycle:**
 - The CPU writes data to a specified memory location.
3. **I/O Read Cycle:**
 - The CPU reads data from an I/O port.
4. **I/O Write Cycle:**
 - The CPU writes data to an I/O port.
5. **Interrupt Acknowledge Cycle:**
 - The CPU acknowledges an interrupt request and retrieves the interrupt vector.

Factors Affecting Bus Cycle Timing:

1. **Bus Speed:**
 - Determined by the clock rate, which defines how quickly data can be transferred.
2. **Wait States:**
 - Delays introduced when slower devices cannot respond within a single clock cycle.
3. **Bus Width:**
 - The size of the data bus determines how much data can be transferred in a single cycle (e.g., 8-bit, 16-bit, or 32-bit).
4. **Arbitration:**
 - If multiple devices request the bus, arbitration determines which device gets access, potentially introducing delays.

Bus Cycle Efficiency:

- Modern systems optimize bus cycles with techniques like burst transfers, pipelining, and caching to reduce latency and improve throughput.

Applications:

- Bus cycles are fundamental to operations like data fetching for CPU instructions, memory access, and device communication, ensuring seamless system functionality.

Basics of DMA (Direct Memory Access)

DMA is a technique that allows peripherals (I/O devices) to transfer data directly to and from memory without the continuous involvement of the CPU. This approach bypasses the CPU for data movement tasks, enabling faster and more efficient data transfers.

Key Components of a DMA System:

1. **DMA Controller:** A hardware module that manages the DMA process.
2. **Channels:** Dedicated paths for data transfer, each associated with a specific device.
3. **Control Signals:** Signals like DMA Request (DREQ) and DMA Acknowledge (DACK) are used for communication between the device and the DMA controller.
4. **Data and Address Bus:** Used by the DMA controller to access memory and transfer data.

Features of DMA

1. **CPU Offloading:**
 - Relieves the CPU from handling data transfer tasks.
 - Allows the CPU to focus on other computational tasks, improving system efficiency.
2. **High-Speed Data Transfer:**
 - Transfers data at higher speeds than CPU-driven methods like programmed I/O or interrupt-driven I/O.
3. **Block Data Transfer:**
 - Transfers data in blocks rather than one byte/word at a time, making it suitable for large data sets.
4. **Minimal CPU Interruption:**
 - The CPU is only interrupted after the completion of a data transfer or in case of errors, reducing overhead.
5. **Automatic Address Management:**
 - The DMA controller automatically increments or decrements the memory address during a data transfer.
6. **Bidirectional Data Transfer:**
 - Supports both memory-to-device and device-to-memory transfers.
7. **Priority Handling:**
 - Allows multiple DMA channels with different priorities, ensuring high-priority devices get preference.
8. **Supports Multiple Devices:**
 - Can handle multiple devices by assigning a dedicated DMA channel to each.
9. **Efficient Peripheral Operations:**
 - Widely used in applications like audio/video processing, disk operations, and network communication where real-time data transfer is essential.

Transfer Modes (determined by mode register of each channel)

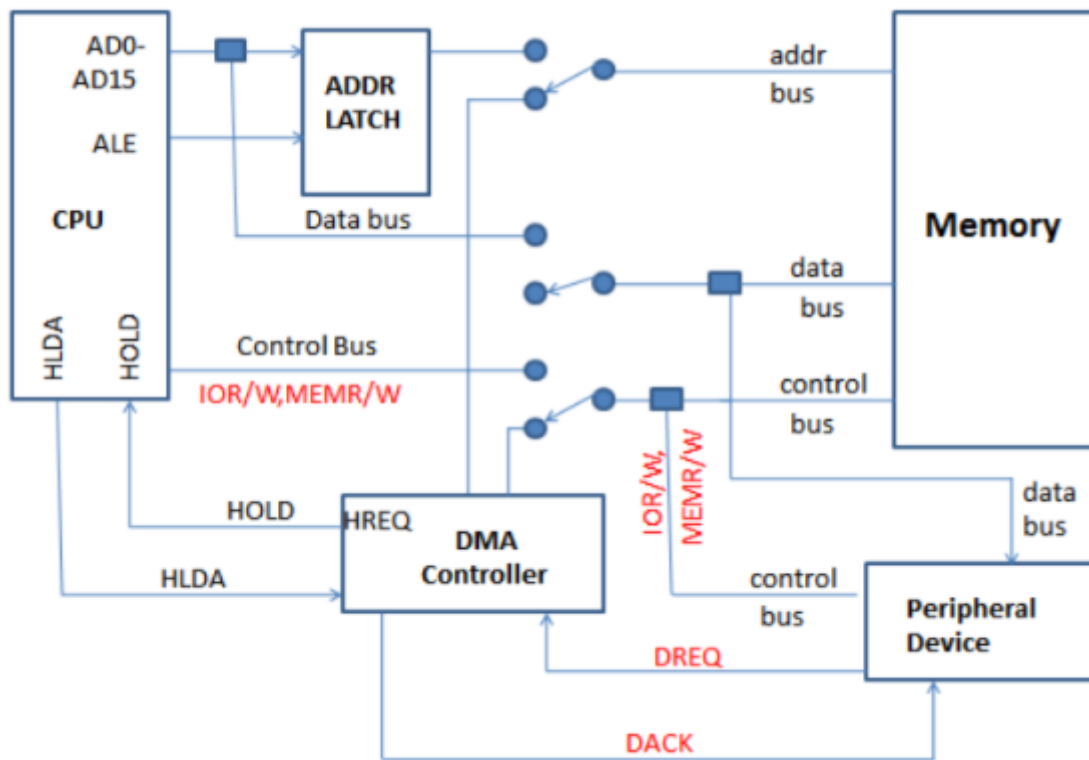
1. **Single:** After each transfer controller releases bus to CPU for at least one bus cycle. Then tests DREQ and as soon as DREQ active steals another bus cycle.
2. **Block:** The bus is released only after entire block transfer.
3. **Demand:** DREQ is checked after each transfer. If inactive waits until it becomes active and resumes the transfer. It allows interface to stop transfer in the event that its device cannot keep up.
4. **Cascade:** This is not a transfer mode – in this mode more than 4 channels can be included by having a slave controller connected to the master.

Applications of DMA:

1. **Audio and Video Data Transfers:** Used for high-speed streaming in multimedia applications.
2. **Disk Operations:** Fast reading/writing of data from storage devices.
3. **Network Interfaces:** High-speed data movement for network communication.
4. **Embedded Systems:** Efficient data transfer in resource-constrained environments like microcontrollers.
5. **Real-Time Systems:** Ensures timely data transfer for time-critical applications.

Steps to perform DMA

1. **Requesting Service:** The external device that requires data transfer activates the **DREQ** (DMA Request) line to request DMA service from the DMA controller.
2. **Bus Request:** If the DMA controller input is not masked (i.e., it's active and ready to process requests), it sends a **bus request** to the microprocessor using the **HOLD** pin, asking for control of the system bus.
3. **Granting Control:** Once the current bus cycle ends, the microprocessor responds by signaling **HLDA** (Hold Acknowledge), granting the DMA controller temporary control. The DMA controller becomes the bus master until it relinquishes this control by deactivating **HOLD**.
4. **Taking Over:** Upon receiving **HLDA**, the DMA controller asserts control over the **Data, Address, and Control buses**.
5. **Address Output:** The DMA controller outputs the content of its **address register** to the address bus, specifying the memory location for the data transfer.
6. **Acknowledging the Device:** The DMA controller sends a **DACK** (DMA Acknowledge) signal to the requesting device, indicating that it can put data on the data bus for transfer.
7. **Activating Read/Write Signals:** The DMA controller also activates the necessary **IOR/IOW** (Input/Output Read/Write) and **MEMW/MEMR** (Memory Write/Read) signals to direct the transfer.
8. **Data Transfer:** Data is transferred from the device to the specified memory address.
9. **Release of Control:** After the transfer is complete, the DMA controller:
 - Releases the **HOLD** request, allowing the microprocessor to regain control of the buses.
 - Releases the buses, returning control to the microprocessor.
10. **Address Register Increment:** The DMA controller increments the **address register** by 1 to prepare for the next byte transfer.
11. **Byte Count Decrement:** The **byte count register** is decremented by 1 to track the remaining bytes.
12. **Loop or End:** If the byte count is non-zero, the process repeats from step 1. If zero, the DMA cycle stops.



Functions of an Address Latch:

1. **Stabilizing Address Signals:**
 - When the CPU generates an address on the address bus, it may need to change that bus quickly for data transfer or other operations. The address latch captures and holds the address signals to ensure the memory or I/O device sees a stable address.
2. **Separating Multiplexed Address/Data Lines:**
 - In many systems (e.g., 8085 or 8086 microprocessors), the address and data share the same physical lines (multiplexed buses) to reduce the number of pins required on the processor. The address latch separates the address from the data, storing the address during the first clock cycle so that the bus can later be used for data transfer.
3. **Driving Address Lines:**
 - The latch provides the necessary drive strength to propagate the address to external memory or peripheral devices, ensuring the signals reach their destination reliably.
4. **Allowing Parallel Operations:**
 - By latching the address, the CPU can perform other tasks while the address latch continues to drive the address to memory or peripherals.

Key Features of an Address Latch:

- **Bidirectional or Unidirectional:** Typically, the latch is unidirectional for address signals (CPU to memory/peripheral).
- **Clock-Controlled:** Captures the address on the rising or falling edge of a clock or strobe signal, such as ALE (Address Latch Enable) in 8085 or 8086.
- **Buffering:** Acts as a buffer to isolate the CPU from the external bus.

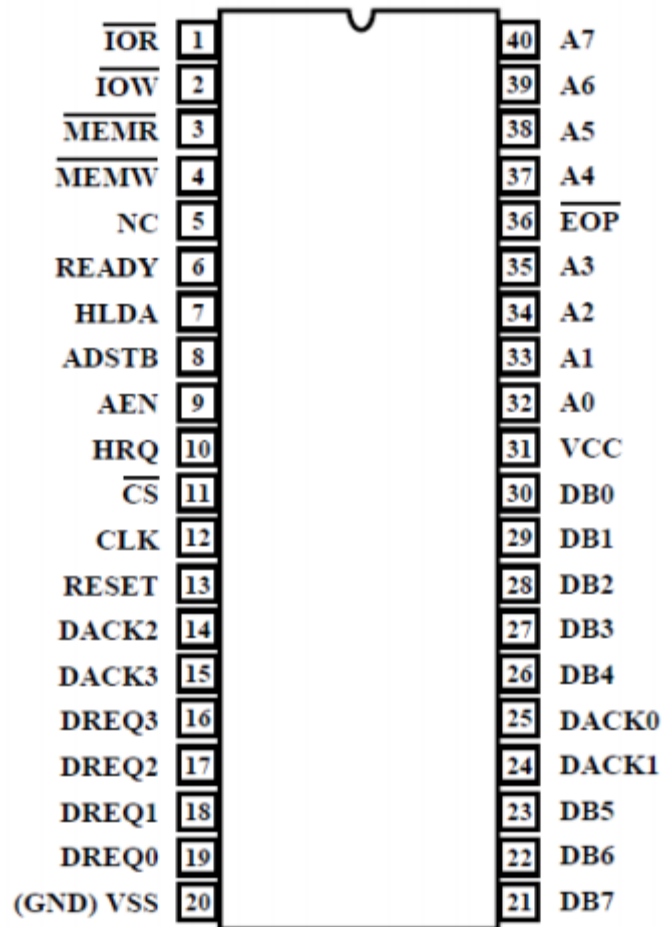


Figure 1: Pin Diagram of 8237 DMA controller

Explanation of 8237 DMA Controller Operations

Address Handling by DMA Controller:

1. **Lower Address Byte (A7-A0):**
 - The DMA controller places the lower 8 bits of the address on the **A7-A0** lines directly.
2. **Higher Address Byte (A15-A8):**
 - The upper byte of the address is placed on **DB7-DB0** (data bus).
 - After setting the Address Enable (AEN) signal high, an **ADSTB (Address Strobe)** pulse is generated to strobe the upper byte into an external address latch (e.g., 8282), which drives the **A15-A8** lines.
3. **Disabling CPU Address Lines:**
 - The DMA controller disables the CPU latches connected to **A19-A8** and **AD7-AD0**, ensuring the controller has full control of the address and data buses.
4. **Upper 4 Address Bits (A19-A16):**
 - The highest 4 address bits cannot be supplied by the 8237 directly.
 - These bits must be provided by a **4-bit I/O port** or hardwired. This port must be pre-initialized with the upper address bits before the DMA transfer begins.

Command and Control Signals:

1. Read/Write Commands:

- The DMA controller generates signals for memory and I/O operations:
 - **IOR** (I/O Read)
 - **IOW** (I/O Write)
 - **MEMR** (Memory Read)
 - **MEMW** (Memory Write)
- These signals replace the **RD, WR, and IO/M** signals of the CPU during DMA transfers.

2. AEN Signal (Address Enable):

- When the **AEN** line is high:
 - The DMA controller takes over the buses, and the CPU's output is disabled.
 - Ensures bus arbitration, preventing contention between the CPU and DMA controller.

End of Process (EOP) Signal:

1. Bidirectional Nature:

- A negative pulse on **EOP** indicates the completion of a block transfer:
 - Can be used as an interrupt to signal the CPU.
- A device can also send a low signal to **EOP** to request suspension of the DMA transfer.

2. Suspension:

- When **EOP** is low, the block transfer is suspended.
- Depending on the operating mode of the 8237, the transfer can resume later.

Other Key Signals:

1. Ready Signal:

- Used for slow peripherals to introduce **wait states**, extending bus cycles and ensuring synchronization.

2. Reset Signal:

- Clears all internal control registers, status registers, temporary registers, and request flags.
- Sets all mask flags, effectively stopping any ongoing DMA operations and reinitializing the controller.